One hierarchy, many shifts

The PCAIR diagonal update: where it fits in the literature,
which fields could use it, first capability studies, and a paper plan

Assessment for PFLARE (branch `claude/dt-bands-diag-update-docs`)

November 2026

Summary

- **What is new.** PCAIR builds an AIR hierarchy for A once. When the matrix becomes $A + D$ with D diagonal, PCAIR updates that hierarchy in place, with $O(\text{nnz})$ work and no matrix–matrix products. It sets the level diagonals, rescales the F-columns of R, lumps $d_{l+1} = R_l d_l$ down the levels, and recomputes or restores the smoother polynomials.
- **It fills a gap in the literature.** Cheap updates under a diagonal shift exist for one-level preconditioners: incomplete factorisations and approximate inverses [13, 14, 15, 16, 17, 18]. AMG reuse schemes keep P (and sometimes R), but they always recompute the Galerkin products [10, 12]. The literature search found no in-place, SpGEMM-free update of a nonsymmetric multigrid hierarchy for a general diagonal, and nothing specific to AIR. The closest relative is lattice-QCD multigrid, which builds once at the critical (hardest) mass and reuses the hierarchy for heavier masses [29]. There the shift is a scalar and the coarse update is trivial.
- **Where to build is the key rule.** Build the hierarchy at the *least diagonally dominant* operator, i.e. the streaming limit or the largest Δt, so every later change increases the diagonal. Our studies confirm it clearly. Built at $s = 0$, every shift from 0 to 10^3 took 2–8 iterations. Built at $s = 1$ or $s = 10^3$, the smaller shifts failed to converge in 300 iterations (Fig. 1). For upwind (M-matrix) operators there is a short argument for why increases are safe (Prop. 1).
- **First capability studies (CPU, CI image).** Problems: 2D/3D upwind advection with constant, curved and recirculating flow, removal contrasts up to 10^4, advection–diffusion, multi-species transport over a growing time step, Radau IIA stage shifts, and 4 MPI ranks.
  - Built at the smallest shift, every update converged: 3.6–6.1 iterations on average, against 2.2–4.8 for a rebuild.
  - An update cost 7–20 times less than a rebuild setup (6.7–20.2).
  - End-to-end it was 1.4–3.4 times faster on one CPU core.
  - With stored or kept smoother coefficients, the update cost fell a further 4–9 times with identical iteration counts.
  - The frozen hierarchy averaged 182 iterations and 9 of its 16 solves did not converge.
- **The GPU case is stronger.** On LUMI one update was about 10 times cheaper than a rebuild, and about 90% of the update was host-side polynomial work. Bands or kept coefficients remove that work. This has not yet been measured on a GPU.
- **Is there a paper? Yes.** We recommend a methods paper framed as *"one reduction-multigrid hierarchy for a whole family of shifted transport operators, on GPUs"*. Its claims would be:

  (i) the update algorithm and its cost model;

(ii) an analysis explaining the "build at the least dominant operator" rule;

(iii) reduction-free updates using coefficient bands;

(iv) a new *multi-shift block apply* that combines the diagonal update with the existing multi-RHS capability: one V-cycle over k columns, each column with its own diagonal;

(v) applications at scale (multigroup Boltzmann, adaptive time stepping, multi-species transport, IRK).

Item (iv) is not implemented yet. It is what would make the paper clearly more than "a reuse option".

1 The capability in one page

We solve a sequence $(L + D_k)x_k = b_k$ with L fixed and D_k diagonal. PCAIR keeps the CF splitting, $A_{ff}^{(l)}$, $A_{fc}^{(l)}$, $R_l = [Z_l\ I]$ and $P_l = [W_l; I]$ on each level. Given $d_1 = \mathrm{diag}(D) - \mathrm{diag}(D_{\mathrm{base}})$, it does the following on each level l:

1. adds $d_l|_F$ (and $d_l|_C$ with C-point smoothing) to the stored diagonals;
2. rebuilds the smoother $M_{ff} \approx A_{ff}^{-1}$ on its fixed sparsity, or simply re-uses stored polynomial coefficients;
3. rescales $Z_l \leftarrow Z_l^{(A)} \mathrm{diag}(\min(1, a_{ff}/(a_{ff} + d_f)))$, which is exact when A_{ff} is diagonal;
4. passes $d_{l+1} = R_l(d_l \circ P_l\mathbf{1})$ to the next level, which is just $R_l d_l$ for a one-point P_l.

The implied coarse operator is $\mathrm{drop}(R_l^{(A)} A_l P_l) + \mathrm{diag}(d_{l+1})$ in place of the Galerkin $R_l(A_l + D_l)P_l$. An update costs about 1–4 V-cycles; it is about 1–2 V-cycles when coefficients are kept. There is no SpGEMM, no repartitioning and no PCMG rebuild.

Why build at the least diagonally dominant operator. There are three reasons. First, the rescaling only ever shrinks Z, and it is exact for a diagonal A_{ff} whenever $d \geq 0$. Second, diagonally scaled smoother polynomials see their spectrum contract towards 1 when the diagonal grows: $D_{ff}'^{-1} A_{ff}' = I + S(D_{ff}^{-1} A_{ff} - I)$ with $0 < S \leq I$. Third, lumping preserves signs:

Proposition 1 (Increases stay increases on every level)**.** *Suppose that on every level $A_{cf} \leq 0$ and the approximation of A_{ff}^{-1} used to build Z_l is entrywise non-negative, so that $Z_l \geq 0$. Suppose also that $P_l \geq 0$ (e.g. the one-point prolongator) and $a_{ff} > 0$. Then $d_1 \geq 0$ implies, for every level l: $d_l \geq 0$; the rescaling factors lie in $(0, 1]$; and the updated R_l stays non-negative.*

Proof sketch. By induction. If $d_l \geq 0$, the factors $a_{ff}/(a_{ff} + d_f)$ lie in $(0, 1]$, so the rescaled Z_l stays ≥ 0. Then $d_{l+1} = R_l(d_l \circ P_l\mathbf{1})$ is a product of non-negative objects. □

The hypotheses hold for upwind finite differences and first-order finite volumes (M-matrices) when A_{ff}^{-1} is approximated by a Neumann-type series. When the diagonal *decreases*, nothing stops a coarse diagonal from crossing zero. That is exactly the failure seen with DG, where Z has mixed signs, and in single precision. A paper should turn this into a proper perturbation bound on the coarse operator in terms of $\|d\|/\|\mathrm{diag}(A)\|$. Theory exists for shift selection in Helmholtz problems [27, 28], and for setup at the critical mass in lattice QCD [29]. These give the natural comparison: our rule is the advective, nonsymmetric version of "set up at the hardest shift".

2 Where this sits in the literature

AIR lineage. AIR [1, 2] and its later variants [3] are reduction-based nonsymmetric AMG methods for advection. hypre's ℓAIR has been used for S_N transport [4] and space–time DG [5]. PFLARE's AIRG [6] uses fixed-sparsity GMRES polynomials for A_{ff}^{-1}. PMISR-DDC CF splittings

Approach	What is reused / how a new shift is handled	Limitation vs. ours
Rebuild AMG/AIR per system (hypre ℓAIR, PFLARE default, MFEM-style) [1, 2, 6]	Nothing; full setup each time	Setup is the GPU bottleneck [8, 43]
Lagged/frozen preconditioner (SNES lag, JFNK) [9]	Everything, unchanged	Fails for large shifts: 182 its avg, 9/16 failed (Sec. 4)
AMG setup reuse (AMGCL, Demidov; MueLu `reuse`; GAMG reuse interp.) [10, 11, 12]	Keep P (and R); recompute RAP and smoothers	Still SpGEMM; R not adapted to the shift
One-level updates of IC/ILU/AINV for $A+\alpha I$ or $A+D$ [13, 14, 15, 16, 17]; interpolated preconditioners [18]	Cheap algebraic update of factors	Single level; not scalable for transport
Shifted Krylov / recycling [19, 20, 21, 22, 23, 24]	One Krylov space for all shifts (or recycled)	Breaks with shift-dependent preconditioning; complementary
Shifted-Laplacian MG (Helmholtz) [25, 26, 27, 28]	MG of a shifted operator as preconditioner	SPD-like, complex shift; theory of "which shift"
Multi-mass lattice QCD MG [29, 30, 31]	Setup at critical mass, reused for all masses	Scalar shift, orthonormal aggregation, not AIR
PCAIR diagonal update	In-place $O(\text{nnz})$ update of every level; R rescaled; lumped coarse shift; coefficient bands	Decreases and non-diagonal (e.g. DG mass) changes are harder

Table 1: Approaches to sequences of shifted systems.

target a well-conditioned A_{ff}. GPU AIRG weak-scales at 66–101% in the solve but only 47–63% in the setup [7, 8]. That setup gap is the main motivation for avoiding rebuilds.

Many-shift solvers. Fully implicit Runge–Kutta (IRK) methods split into real shifted solves $(\eta_i M + \Delta t L)$ [32, 33, 34, 35]. Southworth et al. used AIR for the advective cases [32], rebuilding it for each shift. ParaDiag and diagonalisation in time [36, 37, 38], and rational or exponential integrators [39, 40, 41], need many shifted solves too, but those shifts are *complex*.

GPUs. hypre had to reformulate the AMG setup as SpGEMMs to port it to GPUs [42, 43]. On GPUs the setup is commonly reported as lagging the solve [43, 8]. Blocking right-hand sides through a multigrid hierarchy gives large GPU gains, for example up to 20× on Frontier in lattice QCD [44, 45]. Batched sparse solvers exist for many small systems that share a pattern [46], but they use simple preconditioners, not AMG.

3 Fields that could benefit

In each case below the operator L (advection, streaming, or a transport matrix) is fixed and only a diagonal changes.
Out of scope for now:

Field	Changing diagonal D_k	Notes
Multigroup neutron / photon transport (S_N, P_N) [47, 48, 4, 6]	$\sigma_{t,g}(x)$ per group (10–100s of groups)	Within-group streaming+removal with scattering lagged. The streaming limit is the base. Also k-/α-eigenvalue searches, where $\sigma_t + \alpha/v$ changes every iteration.
Kinetic equations with speed groups (BGK, linear Boltzmann)	$\nu(v)/\lvert v\rvert$	$\lvert v\rvert L_\omega + \nu = \lvert v\rvert(L_\omega + \nu/\lvert v\rvert)$: speed groups become diagonal shifts of one streaming operator.
Thermal radiative transfer [49, 50, 51]	$\sigma_{a,g} + 1/(c\Delta t)$ + Fleck terms	Both group and time step change the diagonal.
Implicit time stepping with adaptive Δt; pseudo-transient continuation	$M/\Delta t$ (lumped mass)	Build at $\Delta t_{\max}$; steady state is the least dominant limit.
IRK / stage-parallel [32, 33]	$\eta_i M/\Delta t$	One hierarchy for all stages and all steps (real-shift preconditioners of [32]).
Multi-species contaminant / reactive transport (PFLOTRAN, operator splitting) [52, 53]	$(R_i/\Delta t + \lambda_i)M$: retardation, decay	10–100 species share velocity. Identical (R_i, λ_i) gives pure multi-RHS. Different values give a multi-shift solve.
Ocean/atmosphere tracers; transport-matrix method; Newton–Krylov spin-up [54, 55, 56]	$\lambda_i I$ (decay, e.g. radiocarbon), $1/\Delta t$	One transport operator for many tracers. Ocean models already reuse one matrix for many passive tracers.
Parameter sweeps, UQ ensembles of reaction/absorption rates	sampled $\lambda(x;\xi)$	Batched shifts are a natural fit for the multi-shift apply.

Table 2: Applications with a fixed transport operator and a changing diagonal.

- complex shifts (ParaDiag, contour integrals), which would need a complex-arithmetic or real-equivalent formulation;
- changes that are not diagonal (a consistent DG mass matrix, or a changing velocity);
- changes of direction in S_N, since each direction is a different L.

4 First capability studies

Setup. Studies ran in the CI image (`stevendargaville/pflare:latest`) on one core of a 4-core container, except the 4-rank case, so timings are indicative only. The driver is `studies/shifted_air/shift_study.c`, and `run_all.sh` reproduces everything.

Problem. L is first-order upwind advection (plus optional diffusion α) on a 2D/3D grid, with Dirichlet boundaries. The system is $(L + s_k W)x = \mathbf{1}$, where W is a checkerboard "material" with values 1 and a contrast c. The velocity is constant ($\pi/4$), curved (bending from $+x$ to $+y$), or recirculating ("double glazing", with $\alpha = 10^{-3}$).

Shifts. Multigroup runs use $s \in \{0\} \cup \{10^{-4}, \ldots, 10^{3}\}$, 16 shifts in all.

Solver. The options are those recommended in the report: GMRES with rtol $= 10^{-8}$; `fc` smoothing; matrix-free `newton_no_extra` smoothers and coarse solve; diagonally scaled polynomials. The update hierarchy is built at the smallest shift.

Columns of Table 3. "Its" is the average number of iterations per system. "Setup/sys" is the rebuild setup time against the update time per system (the initial build is excluded but is counted

in "Total"). "Total" is the end-to-end time of rebuild over update.

Grid	flow, contrast	Its rebuild	Its update	Setup/sys (ms) rebuild	Setup/sys (ms) update	Setup ratio	Total ratio
2D 400^2	const, 1	3.25	4.25	951	99	9.6	2.24
	const, 10^2	3.12	4.88	771	99	7.8	1.68
	const, 10^4	2.94	5.19	648	97	6.7	1.38
	curved, 1	3.31	4.50	840	78	10.7	2.46
	curved, 10^2	3.12	5.25	715	79	9.0	1.91
	curved, 10^4	3.00	5.50	611	79	7.8	1.61
	recirc., 1	4.81	5.50	924	83	11.1	2.40
	recirc., 10^2	3.62	6.00	761	82	9.3	1.80
	recirc., 10^4	3.50	6.12	644	81	8.0	1.54
3D 50^3	const, 1	2.94	3.88	1876	93	20.2	3.36
	const, 10^2	2.88	4.25	1358	94	14.5	2.39
	curved, 1	3.12	4.00	1468	83	17.7	3.22
	curved, 10^2	3.25	4.69	1100	83	13.3	2.33

Table 3: Multigroup-style sweeps, 16 shifts over seven decades. All updates converged.

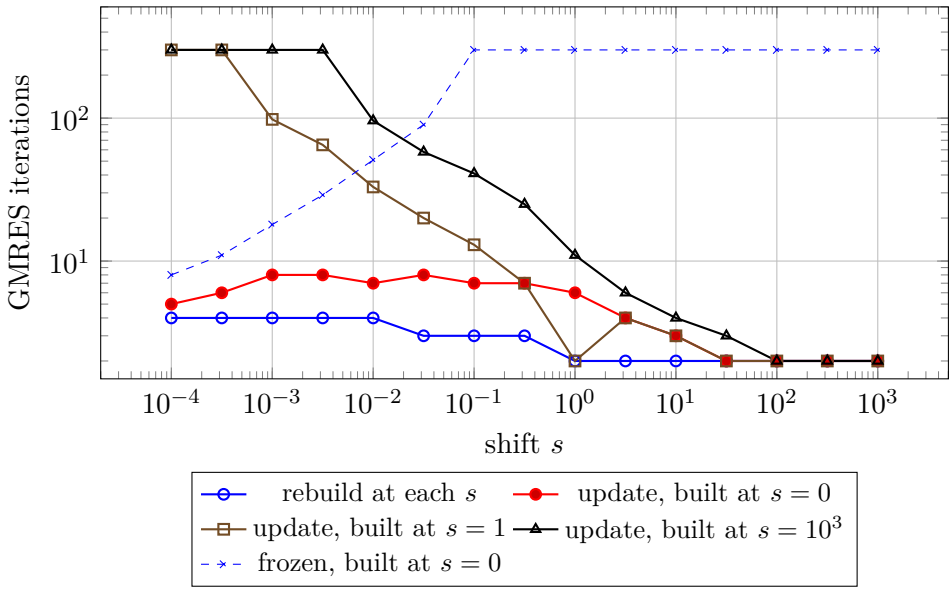


Figure 1: Where to build (200^2, curved flow, contrast 10^2; 300 is the iteration cap). Every update that *increases* the diagonal relative to the build stays at 2–8 iterations. Every large decrease fails.

Findings.

1. **Robust across flows, contrast and dimension.**
   - Every update in Tables 3–4 converged, at 0.9–2.6 more iterations on average than a rebuild.
   - Recirculating flow and contrasts of 10^4 add only about one iteration.
   - Adding diffusion ($\alpha = 0.01, 0.1, 1$; 300^2) gave 5.1, 4.9 and 4.9 iterations against 3.4, 3.4 and 3.8 for a rebuild, with total ratios of 2.1–2.5.
2. **Build where the diagonal is smallest** (Fig. 1).
   - Built at $s = 1$, the update averaged 72 iterations and 3 solves failed. Built at $s = 10^3$, it averaged 109 iterations and 5 failed.

- Built at $s = 0$, it averaged 5.1 iterations.
- The frozen hierarchy averaged 182 iterations, and 9 of 16 solves failed.
- Unscaled polynomials built at 10^3 did better than scaled ones (49 against 109 iterations). This matches the report: scaled polynomials are unsafe for decreases.

3. **Mild growth with refinement.** Curved flow, contrast 10^2, at $n = 100, 200, 400, 800$ (up to 640k unknowns):
   - update: 4.94, 5.06, 5.25, 5.50 iterations;
   - rebuild: 2.94, 2.94, 3.12, 3.31 iterations;
   - setup ratio about 9–10 at every size;
   - total ratio 2.3, 2.1, 2.0, 1.8.
4. **Parallel.** On 4 ranks with $n = 800$ the update took 5.38 iterations, against 5.50 in serial. The total time was 10.7 s against 21.8 s for rebuilding.
5. **The price is a denser hierarchy.** Building at the streaming limit gives a cycle complexity of 35.6 (operator complexity 6.7). A build at $s = 1$ gives 11.1 (2.1), and one at $s = 10^3$ gives 7.2 (1.3). This is why an updated solve costs about twice a rebuilt one on the CPU. One fix is several *anchor hierarchies*, each serving the shifts above it. For $s \geq 1$, updating from an anchor at $s = 1$ cut the solve time from 0.28 s to 0.16 s, with 17 iterations against 21 in total.

Sequence (curved / recirc. flow, 300^2)	systems	Its	update/sys (ms)	Total (s)
6 species (R_i, λ_i) × 25 growing steps, $s = R_i/\Delta t + \lambda_i$				
rebuild every system	150	2.21	309 (setup)	53.7
update, recompute coefficients	150	3.62	40.0	21.6
update, bands $f = 2$ (15 computed, 135 restored)	150	3.62	8.0	15.9
update, keep one set of coefficients	150	3.62	4.6	15.3
Radau IIA(3) real stage shifts $\{3.638, 2.681\}/\Delta t$, 30 growing steps				
rebuild every system	60	2.38	357 (setup)	24.8
update, recompute coefficients	60	4.17	38.0	9.3
update, bands $f = 2$ (10 computed)	60	4.17	10.4	7.6

Table 4: One hierarchy for a whole run: multi-species transport and IRK stages.

Multiple right-hand sides. The study applied `KSPMatSolve` with Richardson to an *updated* hierarchy (400^2). Against solving the columns one at a time, it was 2.0× faster on one CPU core for 4–8 right-hand sides and 1.7× for 64, with the same iteration counts. The GPU gains reported in `docs/gpus.md` are much larger. So updates and block solves already compose: for example, many species with identical (R_i, λ_i), or many sources per group.

Not covered. No GPU runs, no DG or unstructured meshes, and no Boltzmann angle. The report already covers DG time stepping and unstructured meshes. Our results are complementary: they add heterogeneity, 3D, recirculation, species and IRK.

5 Is there a paper?

Yes, with two additions: GPU measurements and some analysis. The ingredients have precedents: one-level shift updates, reusing AMG transfers, and setting up at the hardest shift. No one has combined them into an SpGEMM-free update of a nonsymmetric reduction hierarchy. The application pull is broad (Table 2), and the motivation (setup is the GPU bottleneck) is well documented.

What reviewers will ask, and the answer to have ready.

- *"Isn't this just a reuse option?"* The answer is the analysis (Prop. 1, generalised to a perturbation bound) plus the multi-shift block apply below, which no other AMG offers.
- *Fair baselines.*
  - a PFLARE rebuild;
  - a hypre ℓAIR rebuild;
  - a frozen or lagged hierarchy;
  - a "keep P, R and recompute RAP" reuse (PFLARE's `-pc_air_reuse_sparsity`);
  - for the IRK case, the per-stage AIR of [32].
- *The cost of building at the streaming limit.* The cycle complexity is 2–5× higher. Report work per solve as iterations × cycle complexity, and offer anchors or drop tolerances.
- *Limits.* Decreases, block-diagonal (DG mass) changes and complex shifts. State them plainly, and cite the hybrid lumping prototype for decreases.

6 Recommended paper

Working title. *One hierarchy, many shifts: in-place diagonal updates and multi-shift block solves for reduction multigrid on GPUs.*

Novel contributions.

1. An $O(\text{nnz})$, SpGEMM-free, device-resident update of an AIR hierarchy for $A \to A + D$. It covers the restrictor rescaling, coarse lumping, and smoother rebuild or restore, with a cost model of about 1–4 V-cycles.
2. Analysis:
   - exactness for a diagonal A_{ff};
   - sign and monotonicity preservation for increases (Prop. 1);
   - a bound on the deviation of the coarse operator from Galerkin;
   - the "build at the least diagonally dominant operator" rule, connected to setting up at the critical mass in lattice QCD and to the choice of shift in Helmholtz problems.
3. Reduction-free updates: coefficient bands and kept scaled coefficients, which remove all host work and global reductions from an update on GPUs.
4. **A multi-shift block V-cycle (to implement).** It applies PCAIR to $X \in \mathbb{R}^{n \times k}$ where column j has its own diagonal D_j. All the SpMMs ($A_{ff}X$, $A_{fc}X$, RX, PX) are shared, and the shift enters only as a per-column Hadamard scaling $(A_{ff} + D_j)x_j = A_{ff}x_j + d_j \circ x_j$. The coarse shifts $d_{l+1}^{(j)} = R_l d_l^{(j)}$ form a single SpMM, and the restrictor rescaling is a per-column scaling of the F-rows of the input. Matrix-free Newton smoothers with per-column coefficients cost the same SpMMs. The outer Krylov method can be Richardson, or GMRES batched over columns, with the operator applied as $\mathcal{A}(X) = LX + WX\,\mathrm{diag}(s)$. This turns multigroup sweeps, species and IRK stages into one GPU-saturating solve, and needs no extra storage beyond the dense blocks.
5. GPU results at scale (LUMI): update against rebuild time, and the multi-shift gain over sequential updates. Weak scaling of the update, which has no SpGEMM and no repartitioning, should scale like the solve (66–101%) rather than the setup (47–63%) [8].

Structure (about 25 pages, SISC Methods and Algorithms, or the Software and HPC section; JCP is an alternative).

1. Introduction: sequences of shifted transport systems, why setup dominates on GPUs, contributions.
2. Background: AIR/AIRG, PMISR-DDC, polynomial smoothers.
3. The diagonal update: components, lumping, R rescaling, smoothers.
4. Analysis: exactness, monotonicity, coarse deviation bound, choice of base operator (with a figure like Fig. 1).
5. Avoiding reductions: kept coefficients and bands.
6. The multi-shift block apply, and its batched Krylov.
7. Implementation in PETSc/Kokkos, with a cost model.
8. Results:
   (a) steady shifted sweeps (heterogeneous, 2D/3D, recirculating);
   (b) adaptive time stepping (FD, DG, unstructured; from the existing report);
   (c) multigroup Boltzmann with angle in the matrix (PFLARE's home application);
   (d) multi-species transport, using the multi-shift apply;
   (e) IRK stages;
   (f) GPU strong and weak scaling on LUMI.
9. Limitations: decreases (hybrid lumping), DG mass, complex shifts.
10. Conclusions.

Plan.

- **Weeks 1–3:** GPU timings of the existing update with bands and kept coefficients on LUMI; resize in `PCAIRSetPolyCoeffs` so bands work with `newton`.
- **Weeks 3–8:** implement and test the multi-shift block apply on top of the existing `PCMatApply` path.
- **Weeks 6–10:** analysis section; a multigroup Boltzmann test with heterogeneous $\sigma_{t,g}$; IRK and species drivers. The ones in `studies/` are a start.
- **Weeks 10–14:** scaling runs, baselines (hypre ℓAIR rebuild, reuse sparsity, frozen), writing.

Second paper (optional). An application paper, e.g. multi-species contaminant or ocean tracer transport, or time-dependent multigroup radiation transport, in an application journal. It would use the multi-shift block solve as a black box.

Reproducing the studies

The files are in `studies/shifted_air/`: the driver `shift_study.c`, the shift generators `gen_shifts.py`, the runner `run_all.sh`, raw logs in `results/`, and `summary_table.txt`. To run them, build PFLARE in the CI image, then run `make shift_study` and `./run_all.sh` in that directory. The references were gathered by web search and checked against bibliographic records (DBLP, publishers, arXiv), not full texts; check page numbers and DOIs before citing them in a paper.

References

[1] T.A. Manteuffel, J. Ruge, B.S. Southworth. Nonsymmetric algebraic multigrid based on local approximate ideal restriction (ℓAIR). *SIAM J. Sci. Comput.* 40(6):A4105–A4130, 2018.

[2] T.A. Manteuffel, S. Münzenmaier, J. Ruge, B.S. Southworth. Nonsymmetric reduction-based algebraic multigrid. *SIAM J. Sci. Comput.* 41(5):S242–S268, 2019.

[3] A. Ali, J.J. Brannick, K. Kahl, O.A. Krzysik, J.B. Schroder, B.S. Southworth. Constrained local approximate ideal restriction for advection-diffusion problems. *SIAM J. Sci. Comput.* 46(5):S96–S122, 2024.

[4] J. Hanophy, B.S. Southworth, R. Li, T. Manteuffel, J. Morel. Parallel approximate ideal restriction multigrid for solving the S_N transport equations. *Nucl. Sci. Eng.* 194(11), 2020.

[5] A.A. Sivas, B.S. Southworth, S. Rhebergen. AIR algebraic multigrid for a space-time HDG discretization of advection(-diffusion). *SIAM J. Sci. Comput.* 43(5):A3393–A3416, 2021.

[6] S. Dargaville, R.P. Smedley-Stevenson, P.N. Smith, C.C. Pain. AIR multigrid with GMRES polynomials (AIRG) and additive preconditioners for Boltzmann transport. *J. Comput. Phys.* 518:113342, 2024.

[7] S. Dargaville et al. Coarsening and parallelism with reduction multigrids for hyperbolic Boltzmann transport. *Int. J. High Perform. Comput. Appl.* 39(3):364–384, 2025.

[8] S. Dargaville et al. Solving advection equations with reduction multigrids on GPUs. arXiv:2508.17517 (J. Comput. Phys., 2026).

[9] D.A. Knoll, D.E. Keyes. Jacobian-free Newton–Krylov methods: a survey. *J. Comput. Phys.* 193:357–397, 2004.

[10] D. Demidov. Partial reuse AMG setup cost amortization strategy for the solution of non-steady state problems. arXiv:2108.02054, 2021.

[11] D. Demidov. AMGCL: an efficient, flexible, and extensible algebraic multigrid implementation. *Lobachevskii J. Math.* 40(5):535–546, 2019.

[12] MueLu user documentation, `reuse: type` option (Sandia).

[13] G. Meurant. On the incomplete Cholesky decomposition of a class of perturbed matrices. *SIAM J. Sci. Comput.* 23(2), 2001.

[14] M. Benzi, D. Bertaccini. Approximate inverse preconditioning for shifted linear systems. *BIT* 43:231–244, 2003.

[15] D. Bertaccini. Efficient preconditioning for sequences of parametric complex symmetric linear systems. *ETNA* 18:49–64, 2004.

[16] J. Duintjer Tebbens, M. Tůma. Efficient preconditioning of sequences of nonsymmetric linear systems. *SIAM J. Sci. Comput.* 29(5):1918–1941, 2007.

[17] S. Bellavia, D. Bertaccini, B. Morini. Nonsymmetric preconditioner updates in Newton–Krylov methods for nonlinear systems. *SIAM J. Sci. Comput.* 33(5):2595–2619, 2011.

[18] D. Bertaccini, F. Durastante. Interpolating preconditioners for the solution of sequence of linear systems. *Comput. Math. Appl.* 72(4):1118–1130, 2016.

[19] A. Frommer, U. Glässner. Restarted GMRES for shifted linear systems. *SIAM J. Sci. Comput.* 19(1):15–26, 1998.

[20] V. Simoncini. Restarted full orthogonalization method for shifted linear systems. *BIT* 43(2):459–466, 2003.

[21] M. Baumann, M.B. van Gijzen. Nested Krylov methods for shifted linear systems. *SIAM J. Sci. Comput.* 37(5):S90–S112, 2015.

[22] M.L. Parks, E. de Sturler, G. Mackey, D.D. Johnson, S. Maiti. Recycling Krylov subspaces for sequences of linear systems. *SIAM J. Sci. Comput.* 28(5):1651–1674, 2006.

[23] K.M. Soodhalter, D.B. Szyld, F. Xue. Krylov subspace recycling for sequences of shifted linear systems. *Appl. Numer. Math.*, 2014.

[24] K.M. Soodhalter. Block Krylov subspace recycling for shifted systems with unrelated right-hand sides. *SIAM J. Sci. Comput.* 38(1):A302–A324, 2016.

[25] Y.A. Erlangga, C. Vuik, C.W. Oosterlee. On a class of preconditioners for solving the Helmholtz equation. *Appl. Numer. Math.* 50:409–425, 2004.

[26] Y.A. Erlangga, C.W. Oosterlee, C. Vuik. A novel multigrid based preconditioner for heterogeneous Helmholtz problems. *SIAM J. Sci. Comput.* 27(4):1471–1492, 2006.

[27] M.J. Gander, I.G. Graham, E.A. Spence. Applying GMRES to the Helmholtz equation with shifted Laplacian preconditioning: what is the largest shift for which wavenumber-independent convergence is guaranteed? *Numer. Math.* 131:567–614, 2015.

[28] P.-H. Cocquet, M.J. Gander. How large a shift is needed in the shifted Helmholtz preconditioner for its effective inversion by multigrid? *SIAM J. Sci. Comput.* 39(2):A438–A478, 2017.

[29] R. Babich, J. Brannick, R.C. Brower, M.A. Clark, T.A. Manteuffel, S.F. McCormick, J.C. Osborn, C. Rebbi. Adaptive multigrid algorithm for the lattice Wilson–Dirac operator. *Phys. Rev. Lett.* 105:201602, 2010.

[30] A. Frommer, K. Kahl, S. Krieg, B. Leder, M. Rottmann. Adaptive aggregation-based domain decomposition multigrid for the lattice Wilson–Dirac operator. *SIAM J. Sci. Comput.* 36(4):A1581–A1608, 2014.

[31] C. Alexandrou, S. Bacchio, J. Finkenrath. Multigrid approach in shifted linear systems for the non-degenerated twisted mass operator. arXiv:1805.09584, 2018.

[32] B.S. Southworth, O. Krzysik, W. Pazner, H. De Sterck. Fast solution of fully implicit Runge–Kutta and discontinuous Galerkin in time for numerical PDEs, Part I: the linear setting. *SIAM J. Sci. Comput.* 44(1):A416–A443, 2022.

[33] W. Pazner, P.-O. Persson. Stage-parallel fully implicit Runge–Kutta solvers for discontinuous Galerkin fluid simulations. *J. Comput. Phys.* 335:700–717, 2017.

[34] G.A. Staff, K.-A. Mardal, T.K. Nilssen. Preconditioning of fully implicit Runge–Kutta schemes for parabolic PDEs. *Model. Identif. Control* 27(2):109–123, 2006.

[35] M.M. Rana, V.E. Howle, K. Long, A. Meek, W. Milestone. A new block preconditioner for implicit Runge–Kutta methods for parabolic PDE problems. *SIAM J. Sci. Comput.* 43(5):S475–S495, 2021.

[36] Y. Maday, E.M. Rønquist. Parallelization in time through tensor-product space–time solvers. *C. R. Math.* 346:113–118, 2008.

[37] M.J. Gander, J. Liu, S.-L. Wu, X. Yue, T. Zhou. ParaDiag: parallel-in-time algorithms based on the diagonalization technique. arXiv:2005.09158, 2020.

[38] E. McDonald, J. Pestana, A. Wathen. Preconditioning and iterative solution of all-at-once systems for evolutionary PDEs. *SIAM J. Sci. Comput.* 40(2):A1012–A1033, 2018.

[39] M.J. Gander, S. Güttel. PARAEXP: a parallel integrator for linear initial-value problems. *SIAM J. Sci. Comput.* 35(2):C123–C142, 2013.

[40] S. Güttel. Rational Krylov approximation of matrix functions. *GAMM-Mitt.* 36(1):8–31, 2013.

[41] L.N. Trefethen, J.A.C. Weideman, T. Schmelzer. Talbot quadratures and rational approximations. *BIT* 46:653–670, 2006.

[42] R.D. Falgout, R. Li, B. Sjögreen, L. Wang, U.M. Yang. Porting hypre to heterogeneous computer architectures. *Parallel Comput.* 108:102840, 2021.

[43] R. Li, B. Sjögreen, U.M. Yang. A new class of AMG interpolation methods based on matrix-matrix multiplications. *SIAM J. Sci. Comput.* 43(5), 2021.

[44] P.A. Boyle. Multiple right hand side multigrid for domain wall fermions with a multigrid preconditioned block conjugate gradient algorithm. arXiv:2409.03904, 2024.

[45] B. Krasnopolsky, A. Medvedev. XAMG: a library for solving linear systems with multiple right-hand side vectors. *SoftwareX* 14:100695, 2021.

[46] A. Kashi, P. Nayak, D. Kulkarni, A. Scheinberg, P. Lin, H. Anzt. Batched sparse iterative solvers on GPU for the collision operator for fusion plasma simulations. *IEEE IPDPS*, 2022.

[47] M.L. Adams, E.W. Larsen. Fast iterative methods for discrete-ordinates particle transport calculations. *Prog. Nucl. Energy* 40(1):3–159, 2002.

[48] J.S. Warsa, T.A. Wareing, J.E. Morel. Krylov iterative methods and the degraded effectiveness of DSA for multidimensional S_N calculations in problems with material discontinuities. *Nucl. Sci. Eng.* 147(3):218–248, 2004.

[49] E.W. Larsen, A. Kumar, J.E. Morel. Properties of the implicitly time-differenced equations of thermal radiation transport. *J. Comput. Phys.* 238:82–96, 2013.

[50] A.T. Till, J.S. Warsa, J.E. Morel. Linear multifrequency-grey preconditioning for multigroup thermal radiative transfer. *J. Comput. Phys.*, 2018 (doi:10.1016/j.jcp.2018.06.017; exact title to be checked).

[51] B.S. Southworth, S. Olivier, H. Park, T. Buvoli. One-sweep moment-based semi-implicit-explicit integration for gray thermal radiation transport. *J. Comput. Phys.* 113349, 2024.

[52] C.I. Steefel et al. Reactive transport codes for subsurface environmental simulation. *Comput. Geosci.* 19(3):445–478, 2015.

[53] G.E. Hammond, P.C. Lichtner, R.T. Mills. Evaluating the performance of parallel subsurface simulators: an illustrative example with PFLOTRAN. *Water Resour. Res.* 50(1):208–228, 2014.

[54] S. Khatiwala, M. Visbeck, M.A. Cane. Accelerated simulation of passive tracers in ocean circulation models. *Ocean Modelling* 9(1):51–69, 2005.

[55] X. Li, F.W. Primeau. A fast Newton–Krylov solver for seasonally varying global ocean biogeochemistry models. *Ocean Modelling* 23:13–20, 2008.

[56] A. Bardin, F. Primeau, K. Lindsay. An offline implicit solver for simulating prebomb radiocarbon. *Ocean Modelling* 73:45–58, 2014.